

树——unordered_map 与 C++读取文件夹

同学们，我们朝着用map构建文件夹树 的目标出发了。

一、map（键值对）

为什么存在map？这一点，我们在前面的讲义中也说明了，再次说明一下。

为什么存在map？

假设存在一张成绩表：

学号	姓名	英语	数据结构	数学
1001	张三	100	59.5	62
1002	李四	89	87	99
1003	王五	45	23	0

用结构图去构建，看看下面的代码出现了什么问题？

```
1  typedef struct{
2      int id;
3      char *name;
4      float english;
5      float dataStructure;
6      float math;
7  }Score;
8
9  int main(){
10     Score zhangsan,lisi,wangwu;
11     zhangsan.id = 1001;
12     zhangsan.name = "张三";
13     zhangsan.english = 100;
14     zhangsan.dataStructure = 59.5;
15     zhangsan.math = 62;
16     // 剩下的我就不写了。
17
18     return 0;
19 }
```

上面一段代码中将学号（id）、姓名（name）等全都写到了变量名中。变量名是写到代码中的，并非直观可见。如果写成下面的样式，是不是就好多了？

```

1  {
2      id: 1001,
3      name: "张三",
4      english: 100,
5      dataStructure: 59.5,
6      math: 62
7  }
8  {
9      id: 1002,
10     name: "李四",
11     english: 89,
12     dataStructure: 87,
13     math: 99
14 }
15 {
16     id: 1003,
17     name: "王五",
18     english: 45,
19     dataStructure: 23,
20     math: 90
21 }
22

```

其中，id、name、english等叫作key，1001、“张三”等value。它们放到一起叫作 key-value，中文翻译为键值对。有的语言写作，index-value。道理是一样的。

二、unordered_map的基本概念

unordered_map是C++提供的一种键值对，它是基于哈希函数来建立的——实际上，我们不用关心它的底层逻辑，只需要知道，它的结果是一个map。

```

1
2  #include <iostream>
3  #include <unordered_map>
4  using namespace std;
5
6  int main(){
7
8      unordered_map<string,string> dir;
9
10     dir["id"] = "0";
11     dir["parent_id"] = "-1";
12     dir["name"] = "domain";
13     dir["is_file"] = "false";
14     dir["path"] = "d:\\123\\";
15
16     /**
17     *以上代码实际上得到的map形式如下：

```

```

18     *
19     dir={
20         "id":          "0",
21         "parent_id":   "-1",
22         "name":         "domain",
23         "is_file":      "0",
24         "abs_path":     "D:\\123\\"
25     }
26     */
27     /**map的访问方式如下: */
28     cout<<dir["abs_path"];
29
30     return 0;
31 }

```

对比一下array与map,

```

1 int a[] = {0,1,2,3};
2 cout<<a[0]; //输出0

```

你会发现，map的访问不是别的，它就是将数组中的index变成了key。

但是，在逻辑角度，数组中的元素是“平等”的，元素之间没有本质之分，只有先后顺序之分。

而在map中，每一元素都代表着不同的属性，它们不是“平等”的。

此外，数组是线性的，内存是在一起的。而map的实现是比较复杂，每个元素之间的内存是分开的，因此不能用

`for(int i ;i<size;i++)` 来遍历。

map的遍历如下：

```

1 for (auto it = dir.begin(); it != dir.end(); ++it) {
2     cout<<it->first<<":"<<it->second<<endl;
3     if(it)
4 }

```

其中，it是一个迭代器。相当于指向某个元素的指针，用完了就自动指向下一个。

如果我们非要写成i++的形式，如下：

```

1 auto it = dir.begin();
2 for (int i = 0; i < dir.size(); i++, it++) {
3     cout<<it->first<<":"<<it->second<<endl;
4 }

```

三、升级的unordered_map

在上面的代码中，明明0是整数，我们却用了string类型。这是C和C++这种强类型语言导致的。为了解决这一问题，C++17引入了variant。

```
1  #include <iostream>
2  #include <vector>
3  #include <unordered_map>
4  #include <variant>
5  using namespace std;
6
7  using Variant = variant<int, bool, string>; //相当于定义了一个心得
8
9  int main(){
10
11     unordered_map<string, Variant> dir={
12         {"id", 0},
13         {"parent_id", -1},
14         {"name", "domain"},
15         {"abs_path", "d:\\123\\"},
16         {"is_file", false}
17     };
18
19     cout<<get<string>(dir["abs_path"])<<endl;
20     cout<<get<int>(dir["id"])<<endl;
21     return 0;
22 }
```

C++17还引入了any类型。我知道不少同学，不会主动查找any怎么回事，我就在下面写一下，

```
1  #include <iostream>
2  #include <unordered_map>
3  #include <string>
4  #include <any>
5
6  using namespace std;
7  unordered_map<string, any> config;
8  config["name"] = string("test");    // 字符串
9  config["age"] = 18;                  // int
10 config["ok"] = true;                  // bool
11
12 auto* s = any_cast<string>(&config["name"]);
13 if (s) cout << *s << endl;
14
15 // 读 int
16 auto* i = any_cast<int>(&config["age"]);
17 if (i) cout << *i << endl;
18
```

any表面上看很简单，但它的访问、遍历存在很大问题。我们就不要使用了any，而用variant。

四、读取文件夹

C语言读取文件，应该在C语言课程中学过。

```
1  #include <stdio.h>
2  int main() {
3      FILE *file;
4      int number;
5      file = fopen("example.txt", "r"); // 打开文件用于读取
6      if (file == NULL) {
7          perror("Error opening file");
8          return -1;
9      }
10     // 读取文件直到EOF（文件结束标志）
11     while (fscanf(file, "%d", &number) != EOF) {
12         printf("%d\n", number);
13     }
14     fclose(file); // 关闭文件
15     return 0;
16 }
```

读取文件属于语言的基本操作，因为一般情况下，文件是跨平台的。举个例子，一个图片文件，在Win和mac平台都可以显示。文本文件就更不用说了。

但是，文件夹可不是跨平台的，在win系统中，它的路径如：d:\123\wzd

在Linux、mac、安卓系统中，路径格如 /usr/bin/myfile

因此，在以前的C或者C++中，读取文件夹里的内容（注意不是读取文件——不用fopen函数打开文件）是要引入操作系统的头文件的，如win的写法如下。

```
1  #include <windows.h>
2  #include <stdio.h>
3
4  int main() {
5      WIN32_FIND_DATA FindFileData;
6      HANDLE hFind = INVALID_HANDLE_VALUE;
7      hFind = FindFirstFile("C:\\path\\to\\your\\folder\\*",
8      &FindFileData);
9      if (hFind == INVALID_HANDLE_VALUE) {
10         printf("FindFirstFile failed (%d).\n", GetLastError());
11         return 1;
12     }
```

```

11     }
12
13     do {
14         printf("%s\n", FindFileData.cFileName);
15     } while (FindNextFile(hFind, &FindFileData) != 0);
16
17     FindClose(hFind);
18     return 0;
19 }

```

Linux、mac如下:

```

1  #include <dirent.h>
2  #include <stdio.h>
3  #include <string.h>
4
5  int main(void) {
6      DIR *d;
7      struct dirent *dir;
8      d = opendir("path/to/your/folder");
9      if (d) {
10         while ((dir = readdir(d)) != NULL) {
11             printf("%s\n", dir->d_name);
12         }
13         closedir(d);
14     }
15     return 0;
16 }

```

到了C++17版本后, 读取文件夹内容被标准化了。

```

1  #include <iostream>
2  #include <filesystem>
3  #include <string>
4  /*
5   下面一行代码也是名空间 即 namespace
6   也可以写成 using namespace std::filesystem
7   不过, 一般都写成下面的样子。这样做有个好处, directory_iterator 的归属很明显。
8   */
9  namespace fs = std::filesystem;
10 int main()
11 {
12     // 这是一个路径
13     std::string path = "C:\\Users\\Administrator\\Desktop\\data
14     structure";
15
16     // 遍历这个文件夹下所有的文件
17     /**

```

```

17     auto是个自由变量类型，g++会根据数据自动推导出它的类型
18     &符号，在这里不是取地址，而是“引用”。引用就是给一个变量起别名。大家记住
    这种写法就行了，不想在这里岔开我们的主题太远太远。
19     */
20     for (auto &entry : fs::directory_iterator(path)){
21
22         std::cout << entry.path() << std::endl;    //打印出文件的绝对
    路径
23         std::cout << entry.path().filename() << std::endl; //打印
    出文件名
24         // 判断是文件还是文件夹
25         if (entry.is_regular_file()){
26             std::cout << " : 是文件" << std::endl;
27         }
28         if (entry.is_directory()){
29             std::cout << " : 是文件夹" << std::endl;
30         }
31     }
32     return 0;
33 }

```

五、递归 (Recursion)

我们的目标是读取文件夹树。这里要用到递归。

何为递归？简单点说，就是自己调用自己。

```

1  int main(){
2      cout<<"main函数"<<endl;
3      main();
4      return 0;
5  }

```

再写一个简单的递归

```

1  #include <iostream>
2  using namespace std;
3
4  int add(int n) {
5      if (n == 0) {
6          return 0;
7      }
8      return n + add( n-1 );
9  }
10 int main() {
11     cout<<add(100)<<endl; // 5050
12 }

```

我们用递归实现了累加，书本上用的“累乘”——递归。

六、文件夹访问——深度优先（DFS- Depth-First Search）

我们现在可以这样做，遍历一个文件夹，看到下面有文件夹就进去，直到里面为空或者全是文件为止。

```
1  #include <iostream>
2  #include <filesystem>
3  #include <string>
4
5  namespace fs = std::filesystem;
6
7  void dirTree(std::string path, int deep) {
8      /**如果是文件，就没有子目录，那就返回 */
9      if (fs::is_regular_file(path)) {
10         return;
11     }
12     for (auto &entry: std::filesystem::directory_iterator{path})
13     {
14         /**管他什么，直接打印文件名*/
15         std::cout<<"级别:"<<deep<<"-"<<entry.path().filename()
16         <<std::endl;
17         /**管他什么，直接访问它的子目录*/
18         dirTree(entry.path().string(),deep+1);
19     }
20 }
21
22 int main()
23 {
24     system("chcp 65001");
25     std::string path = "C:\\Users\\Administrator\\Desktop\\data
26     structure";
27     dirTree(path,0);
28     return 0;
29 }
```

是不是超级简单！！

现在，我们不能只满足于将文件、文件夹名字打印出来，而是要封装到map中。

```
1  #include <iostream>
2  #include <filesystem>
```



```

3  #include <string>
4  #include <unordered_map>
5  #include <vector>
6  #include <variant>
7
8  namespace fs = std::filesystem;
9  using namespace std;
10 using var = std::variant<string,int,bool>;
11 vector<unordered_map<string,var>> dirList;
12
13 int id = 0;
14 void dirTree(std::string path, int parent_id) {
15     if (fs::is_regular_file(path)) {
16         return;
17     }
18     for (auto &entry: std::filesystem::directory_iterator{path})
19     {
20         dirList.push_back({
21             {"id",id++},
22             {"parent_id",parent_id},
23             {"path",entry.path().string()},
24             {"name",entry.path().filename().string()},
25             {"is_dir",entry.is_directory()}
26         });
27         dirTree(entry.path().string(), id);
28     }
29 }
30
31
32 int main()
33 {
34     system("chcp 65001");
35     std::string path = "C:\\Users\\Administrator\\Desktop\\data
36     structure";
37
38     /**装载主文件夹*/
39     dirList.push_back({
40         {"id",id++},
41         {"parent_id",-1},
42         {"path",path},
43         {"name","data structure"},
44         {"is_dir",true}
45     });
46
47     //递归调用，深度优先
48     dirTree(path,0);
49
50     //打印测试

```

```

50     for (int i=0;i<dirList.size();i++) {
51         unordered_map<string,var> dir = dirList[i];
52         cout<<get<int>(dir["id"])<<endl;
53         cout<<get<int>(dir["parent_id"])<<endl;
54         cout<<get<string>(dir["name"])<<endl;
55         cout<<get<bool>(dir["is_dir"])<<endl;
56     }
57
58     return 0;
59 }
60

```

七、广度优先 (BFS -- Breadth-First Search)

广度优先，就是遍历一个文件夹下所有的文件（夹），然后放入vector的末尾，主要遍历完这个vector就结束。值得注意的是，这时的vector极其像一个队列（queue）。

```

1
2 #include <iostream>
3 #include <filesystem>
4 #include <string>
5 #include <unordered_map>
6 #include <vector>
7 #include <variant>
8
9 namespace fs = std::filesystem;
10 using namespace std;
11 using var = std::variant<string,int,bool>;
12 vector<unordered_map<string,var>> dirList;
13
14 int id = 0;
15
16 void bfs(std::string path,int parent_id) {
17     if (fs::is_regular_file(path)) {
18         return;
19     }
20     for (auto &entry: std::filesystem::directory_iterator{path})
21     {
22         dirList.push_back({
23             {"id",id++},
24             {"parent_id",parent_id},
25             {"path",entry.path().string()},
26             {"name",entry.path().filename().string()},
27             {"is_dir",entry.is_directory()}
28         });
29     }
30 }

```

```

31 int main()
32 {
33     system("chcp 65001");
34     std::string path = "C:\\Users\\Administrator\\Desktop\\data
    structure";
35
36     dirList.push_back({
37         {"id",id++},
38         {"parent_id",-1},
39         {"path",path},
40         {"name","data structure"},
41         {"is_dir",true}
42     });
43
44     //广度优先式加入到vector
45     for (int i=0;i<dirList.size();i++) {
46         unordered_map<string,var> dir = dirList[i];
47         bfs(get<string>(dir["path"]),get<int>(dir["id"]));
48     }
49
50     //打印测试
51     for (int i=0;i<dirList.size();i++) {
52         unordered_map<string,var> dir = dirList[i];
53         cout<<get<int>(dir["id"])<<endl;
54         cout<<get<int>(dir["parent_id"])<<endl;
55         cout<<get<string>(dir["name"])<<endl;
56         cout<<get<bool>(dir["is_dir"])<<endl;
57     }
58     return 0;
59 }
60

```